

TPractice Questions

- 1. Create a tensor containing numbers 1 to 10.**
- 2. Create a 3×3 matrix tensor.**
- 3. Add two tensors.**
- 4. Multiply two tensors.**
- 5. Create a tensor filled with zeros.**
- 6. Create a tensor filled with ones.**
- 7. Generate random values using TensorFlow.**
- 8. Find the shape of a tensor.**
- 9. Reshape a tensor from (6,) to (2,3).**
- 10. Convert a NumPy array into a TensorFlow tensor.**
- 11. Create a tensor with floating-point values.**
- 12. Find the maximum value in a tensor.**
- 13. Find the minimum value in a tensor.**
- 14. Calculate the mean of tensor values.**
- 15. Build a simple neural network using Keras.**

TensorFlow is an open-source Machine Learning (ML) and Deep Learning (DL) library developed by [Google AI](#).

It is used to:

- Build Artificial Intelligence applications
 - Train Machine Learning models
 - Create Deep Learning neural networks
 - Computer Vision
 - Natural Language Processing (NLP)
 - Speech Recognition
 - Recommendation Systems
-

Why TensorFlow?

TensorFlow helps computers learn patterns from data.

Example

Suppose you have:

	Hours Studied	Marks
1		30
2		40
3		50
4		60

TensorFlow can learn the relationship and predict marks for 5 hours of study.

TensorFlow Architecture

Data



Preprocessing



Model Building



Training



Evaluation



Prediction



Deployment

Installation

```
pip install tensorflow
```

Check version:

```
import tensorflow as tf  
  
print(tf.__version__)
```

TensorFlow Basics

Import TensorFlow

```
import tensorflow as tf
```

Create Constant Tensor

```
a = tf.constant(10)  
b = tf.constant(20)  
  
print(a + b)
```

Output:

30

Create Tensor

1D Tensor

```
tensor = tf.constant([1,2,3,4])  
print(tensor)
```

2D Tensor

```
tensor = tf.constant([  
    [1,2],  
    [3,4]  
)  
  
print(tensor)
```

Tensor vs NumPy

Feature	TensorFlow	NumPy
GPU Support	Yes	No
Deep Learning	Yes	No
Neural Networks	Yes	No
ML Models	Yes	Limited
Speed	Faster on GPU	CPU Based

Tensor Operations

Addition

```
a = tf.constant(10)
b = tf.constant(20)
```

```
print(tf.add(a,b))
```

Subtraction

```
print(tf.subtract(a,b))
```

Multiplication

```
print(tf.multiply(a,b))
```

Division

```
print(tf.divide(a,b))
```

Tensor Shape

```
tensor = tf.constant([
    [1,2,3],
    [4,5,6]
])
```

```
print(tensor.shape)
```

Output:

(2,3)

Reshape Tensor

```
tensor = tf.constant([1,2,3,4,5,6])  
  
reshaped = tf.reshape(tensor,(2,3))  
  
print(reshaped)
```

Random Tensor

```
tensor = tf.random.normal([3,3])  
  
print(tensor)
```

Variables

Variables store trainable parameters.

```
x = tf.Variable(5)  
  
print(x)
```

Update:

```
x.assign(10)  
  
print(x)
```

Keras in TensorFlow

Keras is TensorFlow's high-level API.

Used for:

- Neural Networks
 - Deep Learning
 - Classification
 - Regression
-

Build First Neural Network

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(10, activation='relu'),  
    tf.keras.layers.Dense(1)  
])
```

```
model.compile(  
    optimizer='adam',  
    loss='mse'  
)
```

```
print(model)
```

Layers in TensorFlow

Dense Layer

```
tf.keras.layers.Dense(64)
```

Fully connected layer.

Dropout Layer

`tf.keras.layers.Dropout(0.2)`

Prevents overfitting.

Flatten Layer

`tf.keras.layers.Flatten()`

Converts multi-dimensional data into 1D.

Activation Functions

ReLU

`activation='relu'`

Most common activation function.

Graph:

$$y = \max(0, x)$$

Sigmoid

`activation='sigmoid'`

Output:

0 to 1

Used in Binary Classification.

Softmax

activation='softmax'

Used in Multi-Class Classification.

Model Training

```
model.fit(  
    X_train,  
    y_train,  
    epochs=10  
)
```

Parameters

Parameter	Meaning
epochs	Number of iterations
batch_size	Data processed at once
validation_split	Validation data

Model Evaluation

```
loss = model.evaluate(  
    X_test,  
    y_test  
)
```

Prediction

```
prediction = model.predict(X_test)  
  
print(prediction)
```

Classification Example

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense  
  
model = Sequential([  
    Dense(64, activation='relu'),  
    Dense(32, activation='relu'),  
    Dense(1, activation='sigmoid')  
)  
  
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

CNN (Computer Vision)

CNN = Convolutional Neural Network

Used for:

- Image Classification
- Face Detection
- Medical Imaging

Layers:

Image



Convolution



ReLU



Pooling



Dense



Output

Example:

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(
        32,(3,3),
        activation='relu'
    ),

    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Flatten(),

    tf.keras.layers.Dense(
        10,
```

```
        activation='softmax')
D)
```

RNN (Sequence Data)

Used for:

- Chatbots
- Text Prediction
- Language Translation

```
tf.keras.layers.SimpleRNN(64)
```

LSTM

LSTM remembers long-term information.

```
tf.keras.layers.LSTM(128)
```

Used in:

- NLP
 - Time Series Forecasting
 - Speech Recognition
-

TensorFlow Dataset API

```
dataset = tf.data.Dataset.from_tensor_slices(
    [1,2,3,4,5]
)
```

```
for item in dataset:  
    print(item)
```

Save Model

```
model.save("my_model.h5")
```

Load Model

```
model = tf.keras.models.load_model(  
    "my_model.h5"  
)
```

GPU Check

```
print(  
    tf.config.list_physical_devices(  
        'GPU'  
    )  
)
```

1. What is TensorFlow?

A deep learning and machine learning framework developed by Google.

2. What is a Tensor?

A multi-dimensional array used to store data.

3. What is Keras?

TensorFlow's high-level API for building neural networks.

4. What is ReLU?

An activation function that outputs 0 for negative values and x for positive values.

5. What is CNN?

A neural network mainly used for image processing.

6. What is LSTM?

A special RNN that can remember long-term dependencies.

7. What is Epoch?

One complete pass through the training dataset.

8. What is Batch Size?

The number of samples processed before updating weights.

9. What is Overfitting?

When a model performs well on training data but poorly on new data.

10. What is Dropout?

A technique to reduce overfitting by randomly disabling neurons during training.

TensorFlow Learning Roadmap

Beginner

1. Python Basics

2. NumPy
3. Pandas
4. Matplotlib
5. TensorFlow Basics
6. Tensors
7. Tensor Operations

Intermediate

8. Keras
9. Neural Networks
10. Classification
11. Regression
12. CNN
13. RNN
14. LSTM

Advanced

15. Transfer Learning
16. Object Detection
17. NLP
18. Transformers
19. TensorFlow Serving
20. Model Deployment

Real Projects

- House Price Prediction
- Student Result Prediction
- Spam Email Detection
- Face Mask Detection
- Handwritten Digit Recognition
- Chatbot
- Image Classifier

Nice 👍—here's a **basic TensorFlow practical question set (hands-on programming exercises)** for practice. These are beginner-friendly and build your foundation step by step.

TensorFlow Basic Practical Questions (Hands-on)

1. Installation & Setup

1. Install TensorFlow in your system using pip.
 2. Check TensorFlow version in Python.
 3. Import TensorFlow and print a simple message.
-

2. Tensors Basics

4. Create a scalar tensor.
 5. Create a 1D tensor (vector) with values [1, 2, 3, 4].
 6. Create a 2D tensor (matrix) of shape (2×3).
 7. Convert a Python list into a TensorFlow tensor.
 8. Check the shape, datatype, and dimensions of a tensor.
-

● 3. Tensor Operations

9. Perform addition of two tensors.
 10. Perform subtraction, multiplication, and division on tensors.
 11. Multiply two matrices using TensorFlow.
 12. Find transpose of a matrix using TensorFlow.
 13. Compute element-wise square of a tensor.
-

● 4. Constants & Variables

14. Create a TensorFlow constant.
 15. Create a TensorFlow variable and update its value.
 16. Assign a new value to a TensorFlow variable.
-

● 5. Basic Math Operations

17. Find mean of a tensor.
 18. Find sum of all elements in a tensor.
 19. Find maximum and minimum value in a tensor.
 20. Compute square root of tensor elements.
-

● 6. Random Tensors

21. Create a random tensor using normal distribution.
 22. Create a random tensor using uniform distribution.
 23. Generate a random integer tensor.
-

● 7. Simple Data Handling

24. Convert NumPy array into TensorFlow tensor.
25. Convert TensorFlow tensor back to NumPy array.

8. Basic Neural Network (Very Simple)

26. Create a single neuron (Dense layer) using TensorFlow Keras.
 27. Build a simple sequential model with 1 hidden layer.
 28. Compile a model using optimizer = SGD and loss = MSE.
 29. Train a model on dummy data (X, y).
-

9. Mini Practice Programs

30. Predict output using a trained model.
 31. Build a model that learns $y = 2x + 1$.
 32. Train a model with 10 input samples and check accuracy.
-

Bonus Challenge

33. Build a simple neural network for XOR problem.
34. Train a model to classify even vs odd numbers (basic logic simulation).